

Comparative analysis of DQN and Double-DQN to solve OpenAI Gym CartPole and MountainCar environments

1. Benefits of different parts of DQN architecture:

- a. **Using experience replay:** It breaks the correlation between consecutive steps. By grabbing random past experiences. It helps in well-rounded training and does not overfit.
- b. **Target network:** Naive DQN algorithm has a moving target problem, which makes it harder for the policy to converge. Having a target network, fixes the target, which gets updated after a specific number of training steps.
- c. **Q function:** Tabular Q-values can not solve environments with continuous state, which become solvable with Q function. Also, Q function takes less memory compared to Q-table storage.

2. Environments:

2.1 Gridworld:

- Action Space - {left, right, up, down, pick, drop}
- Observation Space - every single observation is represented by a vector (size 5) having agent position and a boolean flag whether the agent has the package or not, and package location.
- Reward Function - Every action has a reward associated with it.
- Deterministic Environment:
 - Left/Right/Up/Down: (-1)
 - Pick (and if the drone position is the package and first time pick): +25
 - Pick (and if the drone position is the package and 2nd time and onwards pick): -1
 - Pick (and if the drone position is not at the package): -5
 - Drop (and if the drone has package and position is the delivery position): +100
 - Drop (and if the drone does not has a package or position is not delivery position): -5
 - If the drone collide with obstacle: -100

2.2 Cartpole:

There is a pole attached to a cart, and the cart sits on a smooth track. The pole starts upright, but because of gravity, it immediately starts to fall over. To keep it standing, the cart has to move back and forth quickly to stay underneath the pole's center of gravity. The observation space is continuous, with the observation vector having length of four (cartposition, cartvelocity, pole angle, pole angular velocity). The action space is discrete, with a total two number of actions (left, right). The termination condition is that if the cart goes beyond +2.4 or -2.4, or if the pole angle is greater than +12 degrees and -12 degrees, or if the episode length goes beyond 500. Agent gets a reward of +1 for every step before the episode terminates. The agent has to learn to keep the pole upright for as long as possible.

2.3 Mountaincar:

There is a car on a sinusoidal surface. We have the option to choose action from three choices (neutral, left push, right push). The observation space is continuous with the observation vector having length of 2 ([car location, car velocity]). Our aim is to learn a policy to push the car to the position at or greater than 0.5 (at the right top of the sinusoidal surface). The episode terminates after 200 steps or if the car reaches the goal position on top of the hill. By default implementation the agent gets -1 reward for every step taken, but I modified the reward (change in the energy of the car - 0.01). In addition to the change in energy, negative reward of -0.01 is given to make the agent learn to complete in less steps.

3. Neural network architecture:

3.1 Gridworld:

```
net(  
  (fc1): Linear(in_features=5, out_features=64, bias=True)  
  (fc2): Linear(in_features=64, out_features=64, bias=True)  
  (fc3): Linear(in_features=64, out_features=6, bias=True)  
)
```

Since this is a small environment, with observation space size of 2592 and action space of 6. I have used only a small neural network consisting of only fully connected layers.

- **Input layer:** It has 5 neurons, because my observation is in vector form with size 5.
- **Output layer:** It has 6 neurons, because my action space has a size of 6.
- **Activation function:** I have used a linear output layer to keep Q-values unconstrained. I have used Relu activation in the hidden layer.
- **Number of layers:** I have used two hidden layers of size 64, since it is a small environment therefore I have used a small network.

3.2 Cartpole:

```
net_cartpole(  
  (fc1): Linear(in_features=4, out_features=24, bias=True)  
  (fc2): Linear(in_features=24, out_features=24, bias=True)  
  (fc3): Linear(in_features=24, out_features=2, bias=True)  
)
```

- **Input layer:** It has 4 neurons, because my observation is in vector form with size 4.
- **Output layer:** It has 2 neurons, because my action space has a size of 2 (left, right)
- **Activation function:** Similar to the Gridworld environment, I have used a linear output layer to keep Q-values unconstrained. I have used Relu activation in the hidden layer.
- **Number of layers:** I have used two hidden layers of small size 24. Initially I tried using layers of size 128, but it was having an unstable learning, maybe due to overfitting, therefore I have reduced the size of layers.

3.3 Mountaincar:

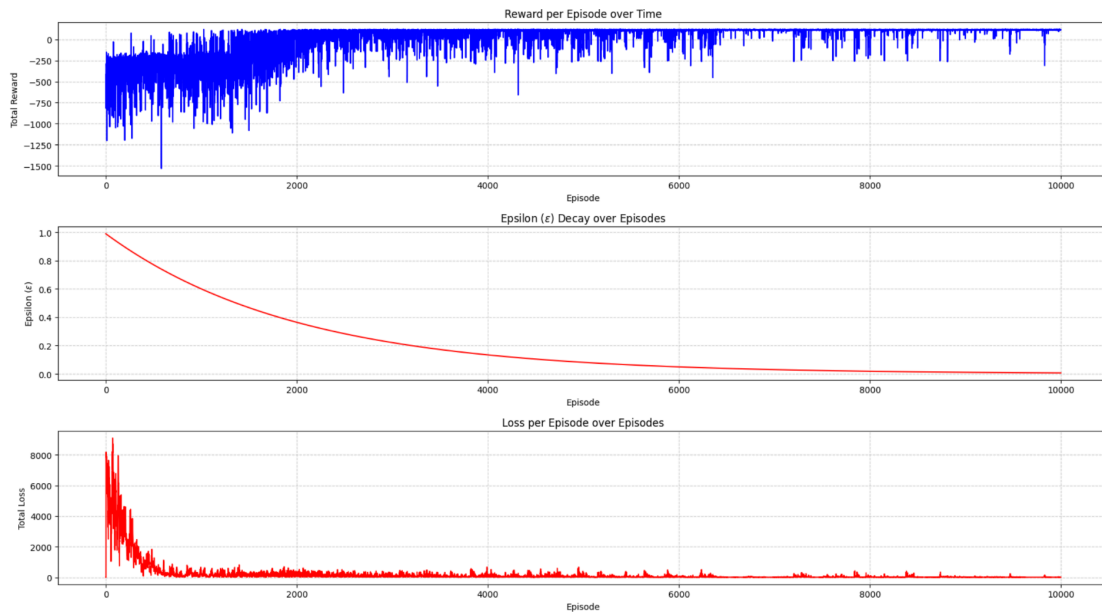
```
net_mountaincar(  
  (fc1): Linear(in_features=2, out_features=128, bias=True)  
  (fc2): Linear(in_features=128, out_features=128, bias=True)  
  (fc3): Linear(in_features=128, out_features=3, bias=True)  
)
```

- **Input layer:** It has 2 neurons, because my observation is in vector form with size 2.
- **Output layer:** It has 3 neurons, because my action space has a size of 3 (neutral, left, right)
- **Activation function:** Similar to the Gridworld environment, I have used a linear output layer to keep Q-values unconstrained. I have used Relu activation in the hidden layer.
- **Number of layers:** I have used two hidden layers of small size 128. I tried solving with smaller layers sizes of 32 and 64, but it was not able to learn the environment, therefore I have to increase the hidden layer size to 128.

4. Training Results and explanation:

4.1 Gridworld Environment

4.1.1 Training graphs



- Reward per episode: It increases with time. And approximately around episode 10000, most of the episodes have positive reward, which means it has converged at around episode number 10000.
- Loss per episode: Loss per episode has a decreasing trend, which makes sense because as the model gets trained, it will be closer to the target network and therefore result in lower loss.

4.1.2 Evaluation result (Greedy policy)

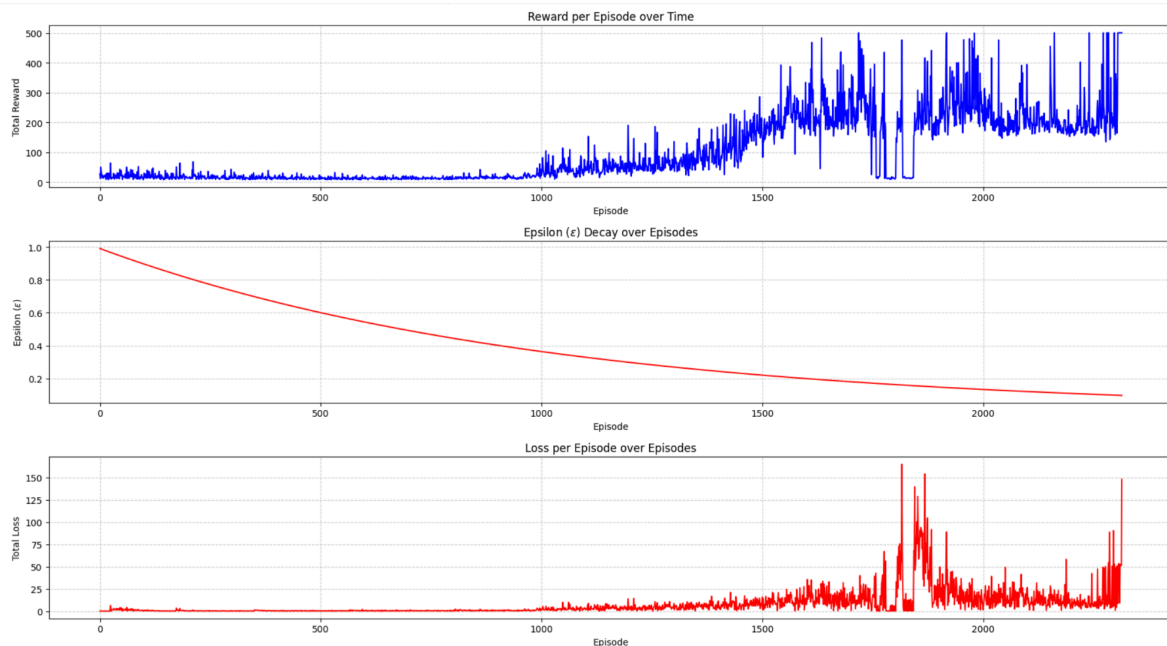
Avg reward : 115.0



4.1.3 Episode demo - Video included in the submission

4.2 Cart-Pole Environment

4.2.1 Training Graphs



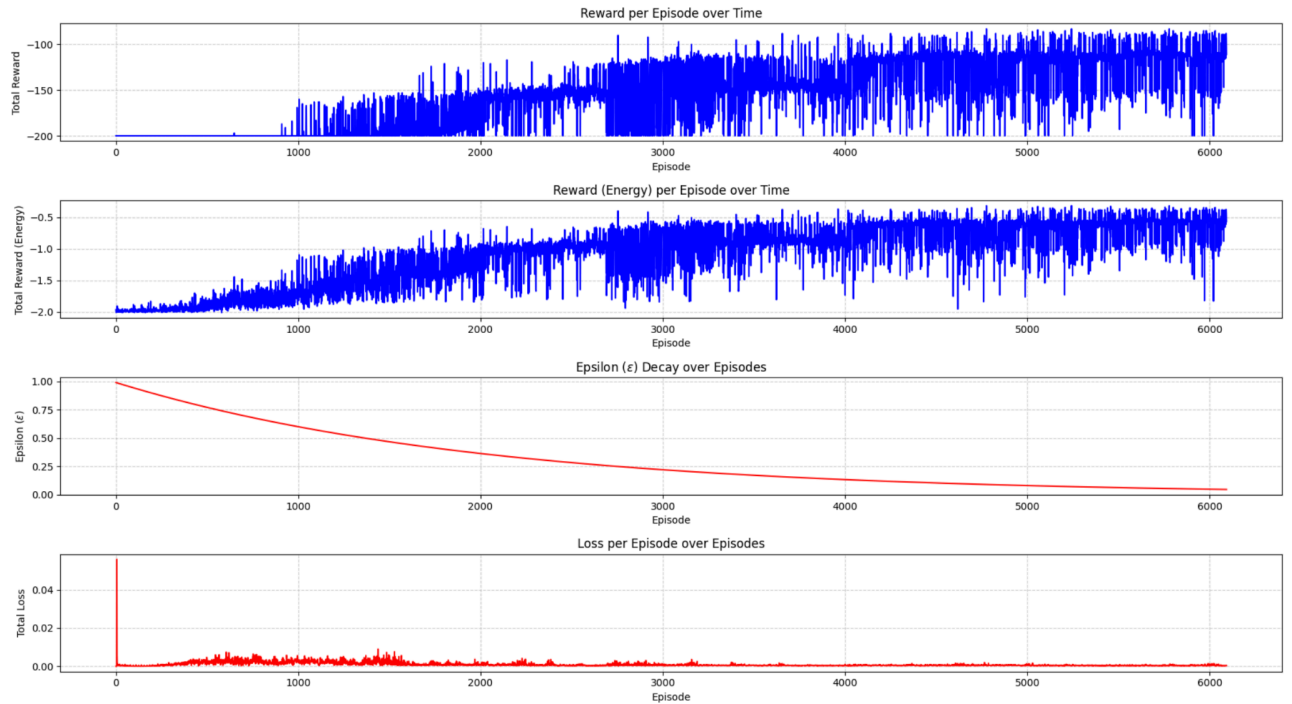
- I have applied early stopping in the training, that is why the training stopped at around 2500 episodes, where it has achieved reward greater than 470 in ten consecutive episodes.
- **Reward per episode:** Reward per episode graph increases with time and gets the desired convergence within 3000 episodes. It shows a dip around episode 1700, this is because of target network sync.
- **Loss per episode:** In this environment loss seems to be increasing, it is because episodes become longer as the network gets better and therefore more training steps in a single episode, which results in higher total loss in a episode. It has a high peak around episode 1700 because of the target network sync and therefore higher loss.

4.2.2 Evaluation result (Greedy policy) on 100 episodes



4.3 MountainCar Environment

4.3.1 Training Graphs



- Compared to checkpoint2 submission, I have updated the reward(energy) function a little bit to get better convergence. Earlier I gave a reward penalty of -0.001 for every step taken, now I have increased this penalty to -0.01 to emphasis more on reducing the number of steps.
- Also I have applied early stopping if 15 consecutive episodes have an average reward(default) of less than -100.
- **Reward(energy) per episode:** Reward(energy) has a continuous increase trend, which finally converges early at around episode number 6000.
- **Reward(default) per episode:** The reward(default) does not increase in the beginning but the reward(energy) is increasing because the agent has learnt to increase the energy reward but has not learned to reach the goal yet. Further, reward(default) increases with time and due to early stopping it converges around episode 6000.
- **Loss per episode:** Loss has increased initially because there would be higher difference between target and policy network, but as the training proceeds the loss gets lower. There are small peaks in the later part of the graph, these are due to the target network sync.

4.3.2 Evaluation result (Greedy policy)

Avg reward : -106.22



5. Saved Model files for DQN algorithm training are provided in the submission

6. Interpretation of the Results:

- For Gridworld I have noticed that tabular methods were able to solve the environment quicker compared to the DQN algorithm. Maybe its because we have a limited number of states, for which it is easier to learn the specific Q-values for different states.
- For a Cartpole environment, DQN with large neural networks gives an unstable learning, because of overfitting. It happens because Cartpole is a small and simple environment.
- For Mountaincar, sparse default reward makes it harder for DQN algo to solve the environment, therefore I had to use the energy based reward which helped the policy to learn constructively in every step.

Algorithm improvement over Vanilla DQN:

1. Double DQN Algorithm

Double DQN works similarly as Double-Q learning, we use two estimators Q_1 (primary network) and Q_2 (target network). And our target value is given by Q_2 , corresponding to the next state and the action having maximum q-value from the primary network, as shown below ($Q_{\theta} = \text{primary network}$, $Q_{\theta'} = \text{target network}$)

$$Q^*(s_t, a_t) = r_t + \gamma Q_{\theta'}(s_{t+1}, \arg \max_{a'} Q_{\theta}(s_{t+1}, a'))$$

Loss is calculated by difference between target and primary q_value as below

$$(Q^*(s_t, a_t) - Q_{\theta}(s_t, a_t))^2$$

But unlike the Double-Q learning, in Double DQN we always do the gradient update in the primary network, using the loss calculated above. And, sync the target and primary network weights after a fixed number of steps.

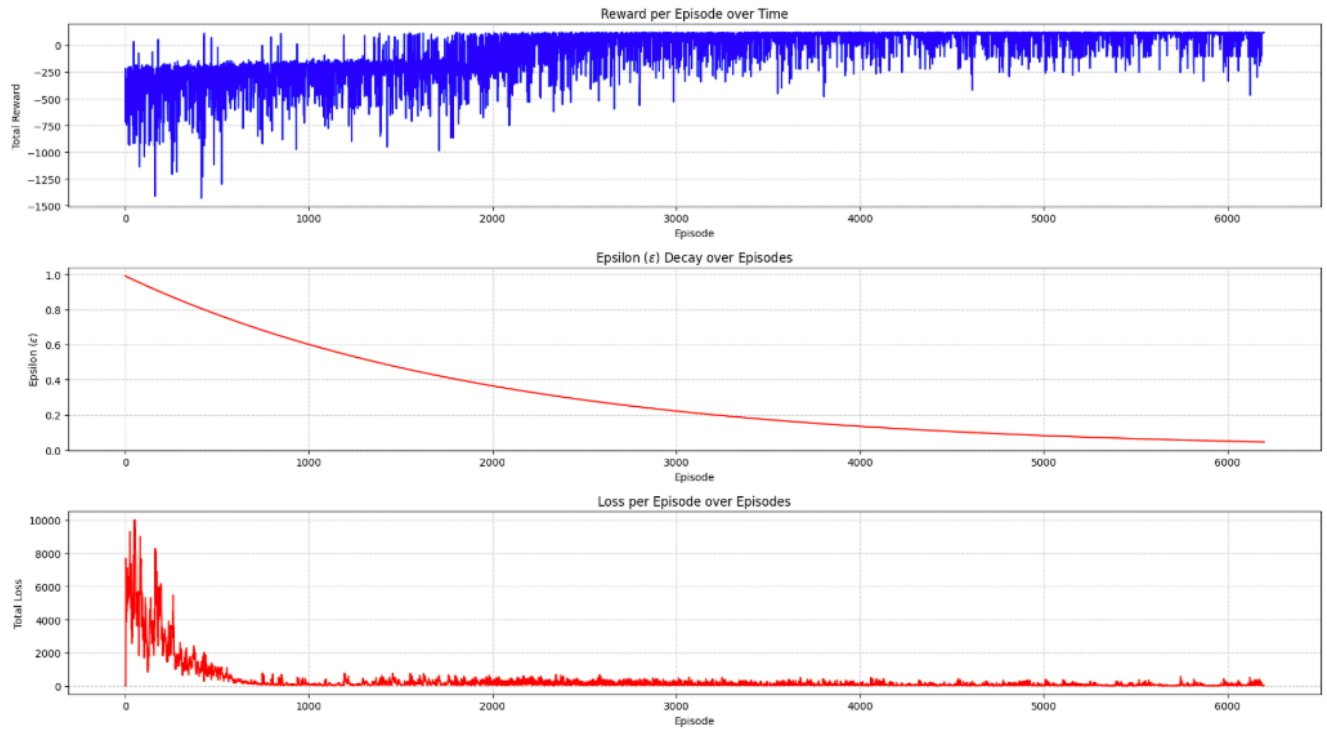
2. Main improvement over vanilla DQN

We use the same two networks in the vanilla DQN and Double DQN. But in the vanilla DQN, the target q-value is given by the maximum q-value output from the target network. But in Double DQN the target q-value is given by the q-value output from the target network, corresponding to the action having maximum q-value output from the primary network for the next_state. Basically, Double DQN tries to solve the overestimation problem.

3. Results:

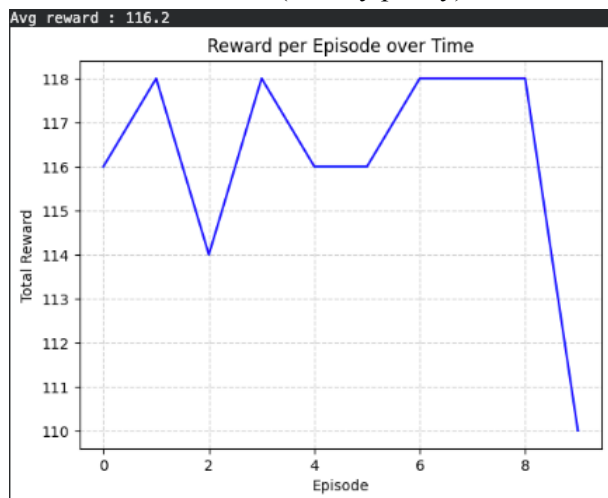
3.1 Gridworld

3.1.1 Training Graphs



- I have applied an early stopping condition that, if I get a reward greater than 110 for ten consecutive episodes, the training stops.
- **Reward per episode:** The graph tells me that in the early stages of training there were higher fluctuations, but later the graph converged at around episode number 6000. There is significant lift visible in the reward graph at around episode number 1500, it is due to target network sync.
- **Loss per episode:** As expected, loss decreases with the time, there are small increases in the middle, it is due to target network sync.

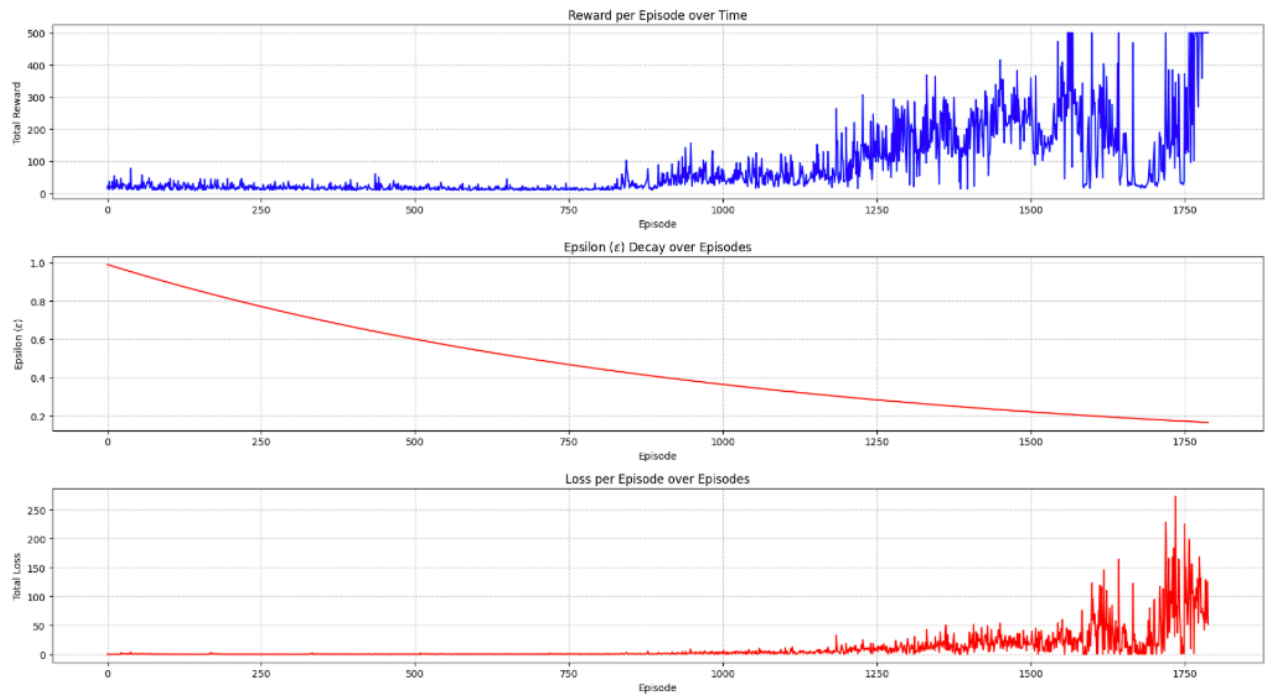
3.1.2 Evaluation result (Greedy policy)



3.1.3 Episode demo - Video included in the submission

3.2 CartpoleV1

3.2.1 Training Graphs



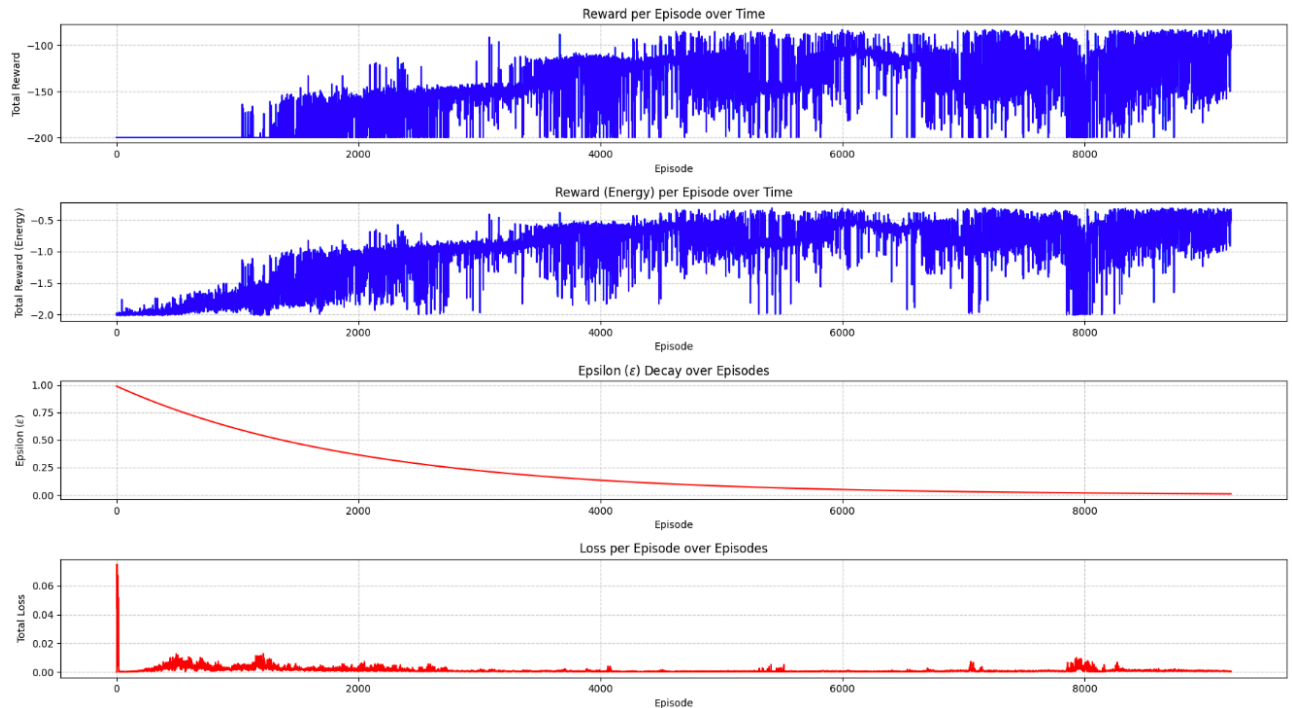
- I have applied a strict early stopping condition. If I get a reward of higher than 470 for ten consecutive episodes, the training stops.
- **Reward per episode:** Reward per episode increases with time. But there is not much learning in the beginning of the training, it is because of the environment setup of cartpole, the episodes end early. But later after episode 700, the reward increases continuously. There is a dip at around episode 1600, it may be due to unstable target sync, but the learning was able to bounce back to the convergence level very quickly.
- **Loss per episode:** losses increase with time, it is because the episode length increases as the agent learns optimal policy and therefore more training steps per episode that leads to higher loss per episode.

3.2.2 Evaluation result (Greedy policy)



3.3 Mountaincar

3.3.1 Training Graphs



- I have used the energy based reward function with a penalty of -0.01 for every step taken. I have also applied an early stopping condition, where the training stops if I get an average reward of above -100 for 15 consecutive episodes.
- **Reward(energy) per episode:** The reward(energy) continuously increases over time, some fluctuations are visible at the end of the training(after episode 6000), it is due to the unstable sync of the target network. But, the learning picks up and eventually converges at around episode 9000.
- **Reward(default) per episode:** The reward(default) graph shows the same trend as reward(energy) as explained above.
- **Loss per episode :** I can see high loss in the beginning, but as expected the loss goes down with training. There are few spikes in the later learning(after episode 6000), it suggests my above understanding that this is because of target network sync. But, further the loss goes down at the end of the training.

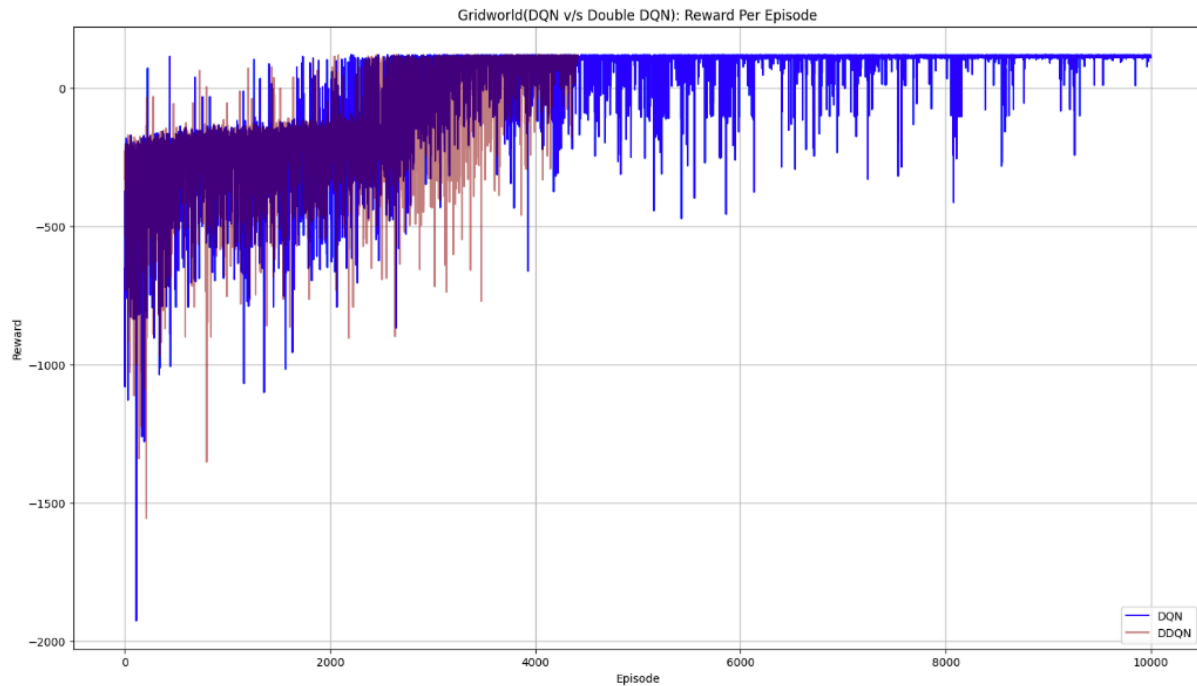
3.3.2 Evaluation result (Greedy policy)



4. Comparison DQN v/s Double DQN (on same environment)

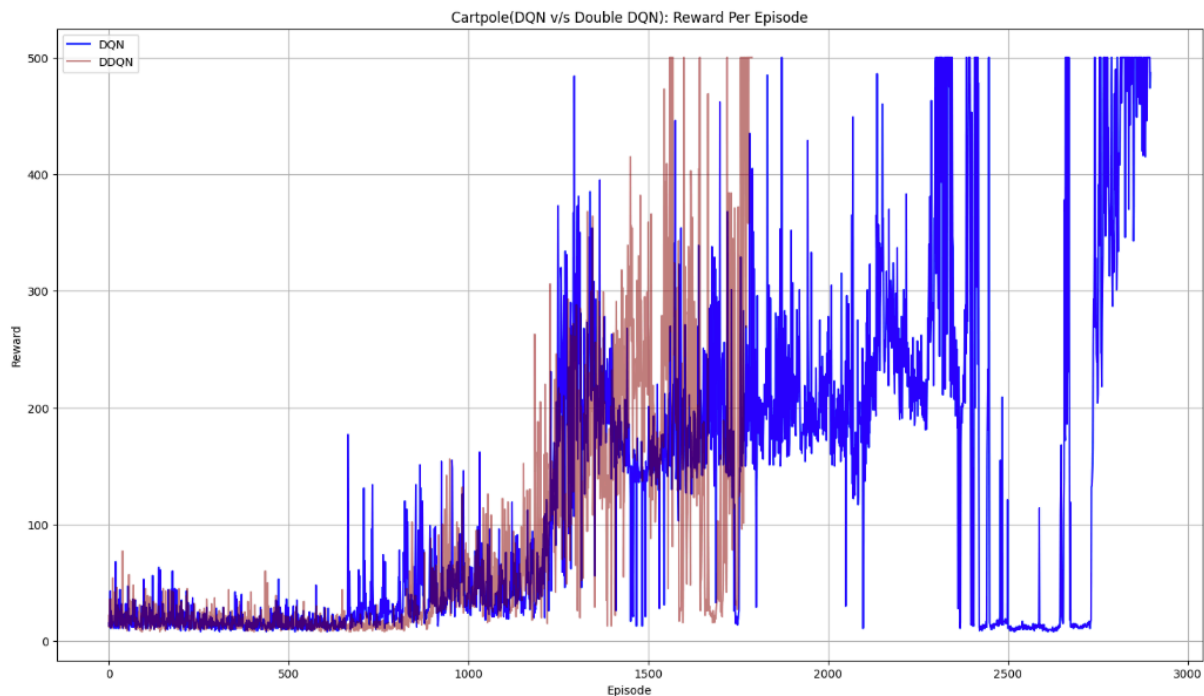
In the graphs below, the blue line is for DQN learning and the orange line is for Double-DQN learning and violet is the overlap region

4.1 Gridworld



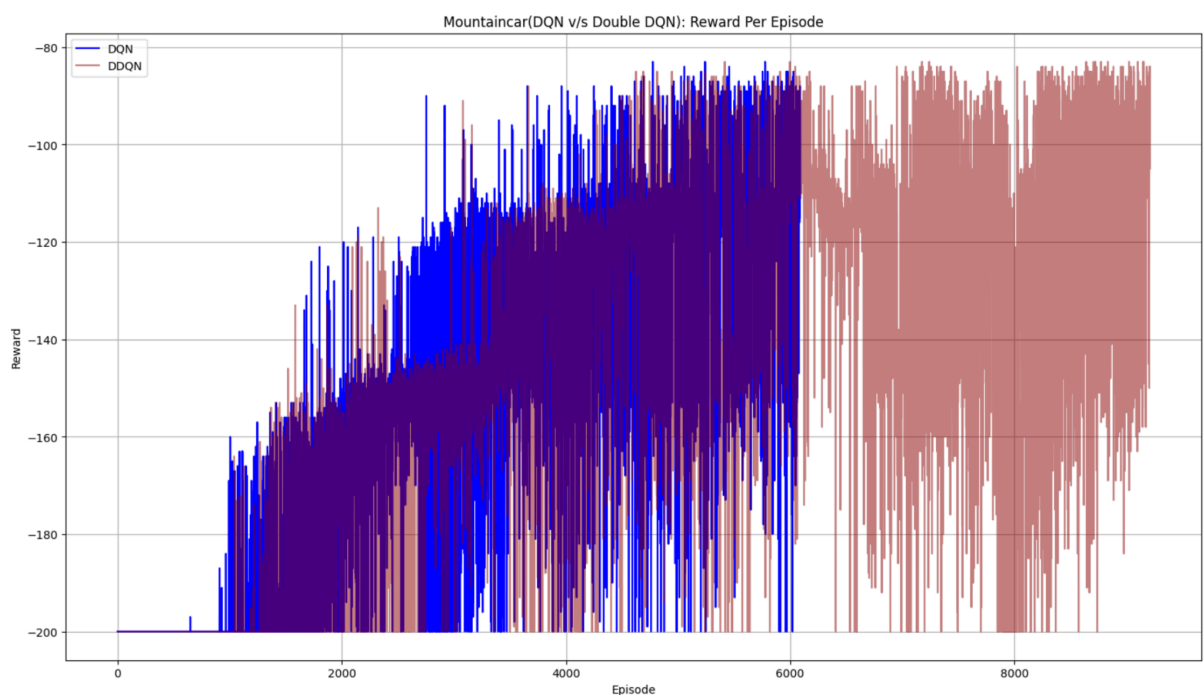
As I can see in the graph, Double DQN converges faster compared to vanilla DQN. In the initial training, vanilla DQN has a little but higher deviated fluctuations compared to double-DQN, and therefore double DQN converges early at around episode number 4000, whereas in vanilla DQN the high fluctuations are visible till episode number 8000.

4.2 CartpoleV1



As I can see in the graph above, the reward in vanilla DQN deviates and gets dipped around episode number 1500, but at the same time the double-DQN has high fluctuations but it still learns in an increasing trend, and further converges early at around episode number 1700. Whereas, the vanilla DQN converges at around episode 3000.

4.3 Mountaincar



As I can see in the graph, the convergence here is early in the DQN(around episode 6000) compared to Double-DQN(around episode 9000). Before episode 6000, the reward is continuously increasing in both algorithms, but Double-DQN is a little bit more stable with less fluctuation compared to DQN, this indicates that double-DQN is giving more stable learning before episode 6000. But after episode

6000, Double-DQN seems to be having network sync instability issues for this particular environment, due to which it takes a little bit longer to converge compared to DQN.

5. Interpretation (same algorithm on different environments and, different algorithms on same environment)

When I applied the same algorithm on gridworld(discrete state) and cartpole/mountaincar(continuous state). Discrete state space environments seem to have more stable learning compared to more fluctuated learning in continuous state space environments.

While observing the impact of different algorithms on the same environment, generally the improved algorithm(Double DQN) seems to have better/stable and faster learning compared to vanilla DQN. Whereas it also depends on the environment, in some cases it can take longer to converge.

6. References

Gymnasium documentation -

https://gymnasium.farama.org/environments/classic_control/mountain_car/

DeepMind Nature paper -

<https://web.stanford.edu/class/psych209/Readings/MnihEtAlHassibis15NatureControlDeepRL.pdf>

Deep RL with Double Q-learning -

<https://arxiv.org/abs/1509.06461>

Python documentation - <https://docs.python.org/3/>

Pytorch Documentation -

https://docs.pytorch.org/tutorials/beginner/blitz/neural_networks_tutorial.html

Numpy Documentation - <https://numpy.org/devdocs/user/index.html>

Lecture slides